

第8回 作業ログ

報告者:24G1021 宇井祐真

報告日: 6月17日(水)

プロジェクト名: マジカル☆観覧車

1 進捗サマリー (要約)

- 全体進捗率: 75[%]
- マイルストーン達成状況: 順調
- 主要成果:
 - － 開始信号の送信確認および BPM・音量変化確認の実施
 - － 開始信号の受信確認および BPM・音量変化の反映確認の実施
 - － 開始信号を楽器ごとにずらして送信するプログラムの作成
 - － 楽器が開始信号を受信して開始するプログラムの作成
- 重大課題:
 - － なし

2 作業ログ (詳細トラッキング)

2.1 概要

作業項目	開始予定日	完了予定日	状態	進捗率	依存関係・ブロッカー	コメント
作業 1 : 開始信号の合図送信確認 (3121: 演奏開始の合図送信確認)	[6/10]	[6/11]	完了	[100%]	[有・開始信号の受信確認に必要]	[なし]
作業 2 : BPM・音量の変化確認 (3122:BPM・音量変化確認)	[6/10]	[6/17]	進行中	[90%]	[有・BPM・音量変化の反映確認に必要]	[音量変化を確認できていないが, BPM と同じ処理のためできると考える]
作業 3 : 開始信号の受信確認 (3211: 開始信号の受信確認)	[6/10]	[6/17]	進行中	[50%]	[無]	[精度保証のための冗長送信にバグを発見]

作業項目	開始予定 日	完了予定 日	状態	進捗率	依存関係・ブロッ カー	コメント
作業 4： BPM・音 量変化の 反映確認 (3212:BPM・ 音量変化 の反映 確認)	[6/10]	[6/17]	進行中	[60%]	[無]	[音量変化の反映を確認して いない]
作業 5： 時間をず らして送 信するプ ログラム の追加 (2223:音 楽・楽器 同期の 実装)	[6/7]	[6/17]	進行中	[80%]	[有・開始信号の 受信確認に必要]	[なし]
作業 6： 楽器で開 始信号を 受け取る プログラ ムの追加 (2223:音 楽・楽器同 期の実装)	[6/7]	[6/17]	進行中	[80%]	[有・開始信号の 受信確認に必要]	[なし]

2.2 日毎の作業

日付	作業項目	作業時間	開始時の状態	終了時の状態	進捗率	課題発生	コメント
6/10	作業 1 3121: 演奏開始の 合図送信確認	1h	進行中	完了	100%	無	-
	作業 2 3122:BPM・音量 変化確認	1h	進行中	進行中	90%	無	BPM 変化は確認済み
	作業 3 3211: 開始信号の 受信確認	2h	進行中	進行中	50%	有	精度保証のための冗長 送信にバグ発見
	作業 4 3212:BPM・音量 変化の反映確認	2h	未着手	進行中	60%	無	観覧車の速度が変更さ れることを確認
	作業 5 2223: 音楽・楽器 同期の実装 (楽器 用の開始送信プ ログラム作成)	0h	進行中	進行中	50%	無	-
	作業 6 2223: 音楽・楽器 同期の実装 (楽器 用の開始受信プ ログラム作成)	0h	進行中	進行中	50%	無	-

日付	作業項目	作業時間	開始時の状態	終了時の状態	進捗率	課題発生	コメント
6/11	作業 1 3121: 演奏開始の 合図送信確認	0h	完了	完了	100%	無	-
	作業 2 3122:BPM・音量 変化確認	0h	進行中	進行中	90%	無	-
	作業 3 3211: 開始信号の 受信確認	0h	進行中	進行中	50%	無	-
	作業 4 3212:BPM・音量 変化の反映確認	0h	進行中	進行中	60%	無	-
	作業 5 2223: 音楽・楽器 同期の実装 (楽器 用の開始送信プ ログラム作成)	0h	進行中	進行中	50%	無	-
	作業 6 2223: 音楽・楽器 同期の実装 (楽器 用の開始受信プ ログラム作成)	0h	進行中	進行中	50%	無	-

日付	作業項目	作業時間	開始時の状態	終了時の状態	進捗率	課題発生	コメント
6/12	作業 1 3121: 演奏開始の 合図送信確認	0h	完了	完了	100%	無	-
	作業 2 3122:BPM・音量 変化確認	0h	進行中	進行中	90%	無	-
	作業 3 3211: 開始信号の 受信確認	0h	進行中	進行中	50%	無	-
	作業 4 3212:BPM・音量 変化の反映確認	0h	進行中	進行中	60%	無	-
	作業 5 2223: 音楽・楽器 同期の実装 (楽器 用の開始送信プ ログラム作成)	0h	進行中	進行中	50%	無	-
	作業 6 2223: 音楽・楽器 同期の実装 (楽器 用の開始受信プ ログラム作成)	0h	進行中	進行中	50%	無	-
6/13	作業 1 3121: 演奏開始の 合図送信確認	0h	完了	完了	100%	無	-
	作業 2 3122:BPM・音量 変化確認	0h	進行中	進行中	90%	無	-
	作業 3 3211: 開始信号の 受信確認	0h	進行中	進行中	50%	無	-
	作業 4 3212:BPM・音量 変化の反映確認	0h	進行中	進行中	60%	無	-
	作業 5 2223: 音楽・楽器 同期の実装 (楽器 用の開始送信プ ログラム作成)	2h	進行中	進行中	65%	無	-
	作業 6 2223: 音楽・楽器 同期の実装 (楽器 用の開始受信プ ログラム作成)	0h	進行中	進行中	65%	無	-

日付	作業項目	作業時間	開始時の状態	終了時の状態	進捗率	課題発生	コメント
6/14	作業 1 3121: 演奏開始の 合図送信確認	0h	完了	完了	100%	無	-
	作業 2 3122:BPM・音量 変化確認	0h	進行中	進行中	90%	無	-
	作業 3 3211: 開始信号の 受信確認	0h	進行中	進行中	50%	無	-
	作業 4 3212:BPM・音量 変化の反映確認	0h	進行中	進行中	60%	無	-
	作業 5 2223: 音楽・楽器 同期の実装 (楽器 用の開始送信プ ログラム作成)	2h	進行中	進行中	70%	無	-
	作業 6 2223: 音楽・楽器 同期の実装 (楽器 用の開始受信プ ログラム作成)	0h	進行中	進行中	70%	無	-
6/15	作業 1 3121: 演奏開始の 合図送信確認	0h	完了	完了	100%	無	-
	作業 2 3122:BPM・音量 変化確認	0h	進行中	進行中	90%	無	-
	作業 3 3211: 開始信号の 受信確認	0h	進行中	進行中	50%	無	-
	作業 4 3212:BPM・音量 変化の反映確認	0h	進行中	進行中	60%	無	-
	作業 5 2223: 音楽・楽器 同期の実装 (楽器 用の開始送信プ ログラム作成)	0h	進行中	進行中	75%	無	-
	作業 6 2223: 音楽・楽器 同期の実装 (楽器 用の開始受信プ ログラム作成)	2h	進行中	進行中	75%	無	-

日付	作業項目	作業時間	開始時の状態	終了時の状態	進捗率	課題発生	コメント
6/16	作業 1 3121: 演奏開始の合図送信確認	0h	完了	完了	100%	無	-
	作業 2 3122:BPM・音量変化確認	0h	進行中	進行中	90%	無	-
	作業 3 3211: 開始信号の受信確認	0h	進行中	進行中	50%	無	-
	作業 4 3212:BPM・音量変化の反映確認	0h	進行中	進行中	60%	無	-
	作業 5 2223: 音楽・楽器同期の実装 (楽器用の開始送信プログラム作成)	0h	進行中	進行中	80%	無	-
	作業 6 2223: 音楽・楽器同期の実装 (楽器用の開始受信プログラム作成)	1h	進行中	進行中	80%	無	演奏部分との結合で100%

3 ガントチャート

作業開始時のガントチャートを図 1, 図 2 に示し, 現在のガントチャートを図 3, 図 4 に示す. また, 全体のガントチャートの進捗情報は, https://docs.google.com/spreadsheets/d/1_nMCHHSSCnY6-YLmoEpx-lQp4S3axuRVMmIirw-6hr8/edit?gid=1575057946#gid=1575057946 で共有している.

No.	作業	担当者	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
2221	通信開始合図の送信実装	宇井 菊池																												
2222	BPM・音量情報送信実装	宇井 菊池																												
2223	音楽・楽器同期の実装	宇井 菊池																												
2224	観覧車の制御	宇井 菊池																												

図 1 実装フェーズのガントチャート (計画)

No.	作業	担当者	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
3121	演奏開始の合図送信確認	宇井 菊池																											
3122	BPM・音量の変化確認	宇井 菊池																											
3141	レーザー受信確認	宇井 菊池																											
3211	開始信号の受信確認	宇井 菊池																											
3212	BPM・音量変化の反映確認	宇井 菊池																											

図2 テストフェーズのガントチャート（計画）

No.	作業	担当者	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
2221	通信開始合図の送信実装	宇井 菊池																											
2222	BPM・音量情報送信実装	宇井 菊池																											
2223	音楽・楽器同期の実装	宇井 菊池																											
2224	観覧車の制御	宇井 菊池																											

図3 実装フェーズのガントチャート（現在）

No.	作業	担当者	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
3121	演奏開始の合図送信確認	宇井 菊池																											
3122	BPM・音量の変化確認	宇井 菊池																											
3141	レーザー受信確認	宇井 菊池																											
3211	開始信号の受信確認	宇井 菊池																											
3212	BPM・音量変化の反映確認	宇井 菊池																											

図4 テストフェーズのガントチャート（現在）

4 課題管理

開始信号の冗長通信の問題は、菊池が対応しているため次回のテストで結合し、再テストを行う。

5 AI 利用状況と検証

AI は使用していない。

6 TRL・TPM

表 1 各開発項目における週次 TRL 進捗

FBS の項目	第 5 週	第 6 週	第 7 週	第 8 週	第 9 週	第 10 週
ずれの少ない同期	1	1	1	1	-	-
可変の速度や音量	1	1	2	2	-	-

表 2 各評価項目における週次 TPM 進捗

評価対象	指標項目	第 5 週	第 6 週	第 7 週	第 8 週	第 9 週	第 10 週
可変抵抗器	入力判定精度	実装中	実装中	40/40	40/40	-	-
加速度センサ	入力判定精度	実装中	39/40	38/40	37/40	-	-
タクトスイッチ	入力判定精度	実装中	40/40	40/40	40/40	-	-
レーザー光受光	入力判定精度	実装中	実装中	実装中	実装中	-	-

7 次回計画

- 目標:
 - 実機を用いて順番に開始信号が送信されることを確認する。
 - マトリクス表示用のプログラムを指揮デバイスのプログラムに結合する。
 - レーザー光の周期から、BPM を算出するプログラムを実機で動作させる。
 - 演奏用のプログラムと結合し、演奏できるようにする。
- 達成基準:
 - シリアルプリントをして到達時刻を確認し、順番に送信されることを確認できる。
 - 指揮デバイスで保持している現在の BPM とマトリクスの BPM が一致している。
 - 観覧車が保持している BPM と同じ BPM を算出できる。
 - 指揮デバイスや観覧車からの情報をもとに演奏ができることを確認する。
- 期限:
 - 6 月 17 日 (水)
 - 6 月 17 日 (水)
 - 6 月 17 日 (水)
 - 6 月 24 日 (水)

8 その他

- 無し

9 付録

9.1 ソースコード

- 添付範囲:

Controller.ino, SyncManager.cpp, SyncManager.h は差分. inst_sync.ino は全体を添付する.

Listing 1 Controller.ino の差分

```
1 uint8_t bpm[5] = {40, 60, 80, 100, 120};
2 bool testFrag = false; // 開始信号が式デバイスで検出されたら true
3 unsigned long before = 0;
4 uint8_t count = 0;
5 unsigned long wait = 0;
6 IPAddress musicIp1(192, 168, 11, 11); // 楽器1のIPアドレス
7 IPAddress musicIp2(192, 168, 11, 12); // 楽器2のIPアドレス
8 IPAddress musicIp3(192, 168, 11, 13); // 楽器3のIPアドレス
9 IPAddress musicIp4(192, 168, 11, 14); // 楽器4のIPアドレス
10 IPAddress ferrisWheelIP2(192, 168, 11, 10); // 観覧車（送信）のIPアドレス cppでも宣言して
    いるため、回避用に2をつけた（テスト）
11
12 void loop() {
13     // char inChar = 0; // シリアル入力がない場合は0（ダミー文字）にしておく
14
15     // // PCからのシリアル入力があれば文字を読み出す
16     // if (Serial.available() > 0) {
17     //     inChar = Serial.read();
18     // }
19
20     // ★シリアル入力があろうがなかろうが、毎ループ常に各関数を呼び出す
21     // sendStart(inChar);
22     // sendBPM(inChar, stepNumber);
23     // sendVolume(inChar, stepNumber);
24     sendStart();
25
26     uint8_t currentBpm = bpm[currentBpmStep];
27
28     unsigned long interval_time = 240000 / currentBpm; // 1小節 = 4拍 * (60000/bpm)
29
30     unsigned long now = millis();
31     if(now >= wait && testFrag){
32         if(now - before >= interval_time){
33             if(!sendStartToMusic()){
34                 testFrag = false; //開始信号を4つの楽器に送信完了
35             }
36             before = now;
37         }
38     }
39 }
```

```

40     if (now - lastSampleTime >= SAMPLE_INTERVAL) {
41         lastSampleTime = now;
42         updatePotentiometers();
43     }
44
45
46     if(now - lastSampleTime2 >= ACC_SAMPLE_INTERVAL){
47         lastSampleTime2 = now;
48
49         currentAccVal = readAccMagnitude();
50         float diff = fabsf(currentAccVal - lastAccVal);
51
52         bool overThreshold = (diff > accThreshold);
53         bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
54
55         if (overThreshold && intervalPassed) {
56
57             lastDetectTime = now;
58
59             isStarted = !isStarted;
60
61             Serial.print("[DETECT]_diff=");
62             Serial.print(diff, 4);
63             Serial.print("_isStarted=");
64             Serial.println(isStarted ? "true_(開始)" : "false_(停止)");
65
66             updateShakedInfo();
67         }
68
69         lastAccVal = currentAccVal;
70     }
71 }

```

Listing 2 SyncManager.cpp の差分

```

1 void SyncManager::sendStart(IPAddress targetIp) {
2     // 設計書通りの2バイト固定長ペイロード (第1バイト:ヘッダ'S', 第2バイト:ダミー 0x00)
3     uint8_t startPacket[2] = {'S', 0x00};
4     packet3(startPacket, 2, targetIp);
5     //packet3(startPacket, 2, musicIP);
6 }

```

Listing 3 SyncManager.h の差分

```

1 class SyncManager {
2 public:
3     // コンストラクタ (ポート番号などを初期化する)
4     SyncManager();
5     // Wi-FiとUDPを準備する関数
6     void begin(const char ssid[], const char pass[]);
7     // 演奏開始合図 ('S') を送信する自作関数
8     void sendStart(IPAddress targetIp);
9     // BPM情報を送信する関数 (引数の型を追加)
10    void sendBPM(uint8_t stepNumber);
11    // 音量情報を送信する関数 (引数の型を追加)

```

```

12     void sendVolume(uint8_t stepNumber);
13
14 private:
15     // 3回冗長送信を行う共通処理 (privateへ移動し、型を定義)
16     void packet3(const uint8_t packet[], uint8_t b, IPAddress targetIP);
17 };

```

Listing 4 inst_sync.ino の全体

```

1  #include <WiFiS3.h>
2  #include <WiFiUdp.h>
3
4  const char SSID[] = "hackathon001-WPA2";
5  const char PASS[] = "hackathon001";
6
7  IPAddress localIP(192, 168, 11, 11); // 各楽器が被らないようにIPアドレスを指定. ドラム
8  //IPAddress localIP(192, 168, 11, 12); // ピアノ
9  //IPAddress localIP(192, 168, 11, 13); // ストリングス
10 //IPAddress localIP(192, 168, 11, 14); // フルート
11
12 const uint16_t LOCAL_PORT = 5005;
13
14 WiFiUDP Udp;
15
16 uint8_t packetBuffer[10];
17
18 const int CDS_PIN = A0;
19
20 const int CDS_ON_THRESHOLD = 800; // レーザー検知とみなす上限値
21 const int CDS_OFF_THRESHOLD = 600; // ON→OFF 判定の下限値
22
23 const unsigned long DEBOUNCE_MS = 200; // 誤検知防止用の待機時間
24
25 const int BPM_LIST[] = { 60, 90, 120, 150, 180 }; // BPMのリスト
26
27 const int MIN_PULSES_FOR_READY = 2; // BPMが安定したとみなす最小パルス回数
28
29 const uint8_t volList[5] = {10, 20, 30, 40, 50}; // 要素は仮 音量のリスト
30
31 class SyncManager {
32 public:
33     SyncManager()
34         : lastDetectTime(0), rawBPM(0.0f), correctedBPM(0),
35           pulseCount(0), laserOn(false) {}
36
37     // ループ毎に呼ぶ。新たに BPM が確定したら true を返す
38     bool update(int cdsValue, unsigned long nowMs) {
39         if (!detectLight(cdsValue)) return false;
40
41         if (nowMs - lastDetectTime < DEBOUNCE_MS) return false; // 誤検知防止
42
43         if (lastDetectTime != 0) {
44             unsigned long interval = nowMs - lastDetectTime;
45             rawBPM = 30000.0f / (float)interval; // レーザー間隔 = 半小節

```

```

46     correctedBPM = rawToDefined(rawBPM);
47     pulseCount++;
48     lastDetectTime = nowMs;
49     return true;
50 }
51
52 // 初回検知：時刻だけ記録
53 lastDetectTime = nowMs;
54 return false;
55 }
56
57 float getRawBPM()          const { return rawBPM; }
58 int  getCorrectedBPM()     const { return correctedBPM; }
59 int  getPulseCount()      const { return pulseCount; }
60
61 private:
62     unsigned long lastDetectTime;
63     float        rawBPM;
64     int          correctedBPM;
65     int          pulseCount;
66     bool         laserOn;
67
68     bool detectLight(int v) {
69         if (!laserOn && v >= CDS_ON_THRESHOLD) { laserOn = true; return true; }
70         if ( laserOn && v <  CDS_OFF_THRESHOLD){ laserOn = false; }
71         return false;
72     }
73
74 // BPMリストから一番近いBPMを選択
75 int rawToDefined(float raw) {
76     int best = BPM_LIST[0];
77     float bestDiff = fabs(raw - BPM_LIST[0]);
78     for (int i = 1; i < 5; i++) {
79         float d = fabs(raw - BPM_LIST[i]);
80         if (d < bestDiff) { bestDiff = d; best = BPM_LIST[i]; }
81     }
82     return best;
83 }
84 };
85
86 // state: 0=BPM推定中, 1=演奏開始待機, 2=演奏中
87 SyncManager syncManager;
88 int         state = 0;
89
90 void onPlayStart(int bpm);
91 void onBeatPulse(int bpm);
92 void onVolumeChange(uint8_t stepNumber);
93
94 void setup() {
95     Serial.begin(115200);
96
97     WiFi.config(localIP);
98     WiFi.begin(SSID, PASS);
99

```

```

100 Serial.print("Connecting to WiFi...");
101 while (WiFi.status() != WL_CONNECTED) {
102     delay(500);
103     Serial.print(".");
104 }
105 Serial.println("\nWiFi Connected!");
106 Serial.print("IP:"); Serial.println(WiFi.localIP());
107
108 Udp.begin(LOCAL_PORT);
109 Serial.print("Listening on UDP port"); Serial.println(LOCAL_PORT);
110 Serial.println("セットアップ完了");
111 }
112
113 void loop() {
114     unsigned long now = millis();
115     int cdsValue = analogRead(CDS_PIN);
116
117     // ---- UDP 受信 (どの state でも受け付ける) ----
118     int packetSize = Udp.parsePacket();
119     if (packetSize >= 2) {
120         Udp.read(packetBuffer, sizeof(packetBuffer));
121         char header = (char)packetBuffer[0];
122         uint8_t payload = packetBuffer[1];
123
124         switch (header) {
125             case 'S':
126                 if (state >= 1) { // BPM確定後のみ受け付ける
127                     state = 2;
128                     Serial.println(F("[START] 'S' 受信 → 演奏開始"));
129                     onPlayStart(syncManager.getCorrectedBPM());
130                 }
131                 break;
132
133             case 'V':
134                 Serial.print(F("[VOL] step=")); Serial.println(payload);
135                 onVolumeChange(payload);
136                 break;
137
138             default:
139                 Serial.print(F("[WARN] 不明なヘッダ:")); Serial.println(header);
140                 break;
141         }
142     } else if (packetSize > 0) {
143         // サイズが想定外のパケットはバッファを空にする
144         while (Udp.available()) Udp.read();
145     }
146
147     // ---- CdS レーザー検知・BPM算出 ----
148     bool newBPM = syncManager.update(cdsValue, now);
149
150     // ---- ステートマシン ----
151     switch (state) {
152
153         case 0: // BPM推定中

```

```

154     if (newBPM) {
155         Serial.print(F("[CAL] pulse=")); Serial.print(syncManager.getPulseCount());
156         Serial.print(F(" rawBPM=")); Serial.print(syncManager.getRawBPM(), 1);
157         Serial.print(F(" correctedBPM=")); Serial.println(syncManager.getCorrectedBPM
158         ());
159
160         if (syncManager.getPulseCount() >= MIN_PULSES_FOR_READY) {
161             state = 1;
162             Serial.print(F("[READY] BPM=")); Serial.print(syncManager.getCorrectedBPM());
163             Serial.println(F("'S' パケット待機中..."));
164         }
165         break;
166
167     case 1: // 演奏開始待機: 引き続きBPMを更新
168         if (newBPM) {
169             Serial.print(F("[READY] BPM更新->")); Serial.println(syncManager.
170             getCorrectedBPM());
171             break;
172
173         case 2: // 演奏中
174             // [instCtrl] 結合時はここで updateTick() を毎ループ呼ぶ
175             //             → 内部チック(curTick)を micros() ベースで自律更新する
176             // [instCtrl] 結合時はここで sendPacket() を毎ループ呼ぶ
177             //             → タイミングが来たノートをシリアルに送信する
178
179             if (newBPM) {
180                 Serial.print(F("[SYNC] レーザー検知 BPM=")); Serial.println(syncManager.
181                 getCorrectedBPM());
182                 onBeatPulse(syncManager.getCorrectedBPM());
183             }
184             break;
185         }
186     }
187
188     // 演奏開始時に1回呼ばれる
189     void onPlayStart(int bpm) {
190         // [instCtrl] sendBpmValue(bpm) → tickIval を BPM から計算して更新
191         //                                     tickIval = 6000000UL / bpm / 16
192         // [instCtrl] curTick = 0 → チックカウンタをリセット (楽譜先頭から)
193         // [instCtrl] scoreIdx = 0 → ノートインデックスを先頭に戻す
194         // [instCtrl] lastTick = micros() → タイマ基準時刻を現在時刻にセット
195     }
196
197     // 演奏中, レーザーパルス検知ごとに呼ばれる (半小節ごと)
198     void onBeatPulse(int bpm) {
199         // [instCtrl] sendBpmValue(bpm) → レーザー間隔から算出した BPM で tickIval を補正
200         //                                     BPMが変わっていた場合のみ実際に更新される
201     }
202
203     // 'V' パケット受信時に呼ばれる
204     void onVolumeChange(uint8_t stepNumber) {
205         // [instCtrl] sendVolumeValue(stepNumber) → 音量をシリアル送信

```


205	//	前回値と変化があった場合のみ送信される
206	}	